TECHNICAL REFERENCE

Forecast & Stock Evaluation Reference

Every threshold, rule, and variable behind the Sales Forecast, Low Stock alerts, FIFO Urgency tags, and Model Accuracy gauges in EggLogan Egg Storage.

EggLogan Egg Storage · K-Nearest-Neighbour demand forecasting · PHP 8 / MySQL

This document explains, in plain language first and exact code second, how the system decides whether you have **enough stock**, how urgently a tray should be sold, and how much the forecast itself can be trusted. It lists every numeric threshold it uses and names every variable in the model — the ones that go in (*independent*) and the ones that come out (*dependent*).

5	7	2	11
STOCK & FORECAST CLASSIFICATIONS	INDEPENDENT MODEL FEATURES	DEPENDENT TARGET VARIABLES	EVALUATION METRICS

Source: `classes/KNNPredictor.php`, `classes/ModelEvaluator.php`, `classes/AlertManager.php`, `includes/functions.php`, `pages/predictions.php`, `pages/picklist.php`

00 Contents

01	How the whole thing works	*the five-step journey from sales history to a verdict*
02	The five classifications & their thresholds	*exact cut-off values, in one place*
03	Independent variables (the inputs)	*the 7 features, explained one by one*
04	Dependent variables (the outputs)	*what the model is actually predicting*
05	Stock & lifecycle variables	*the inputs thresholds use but the model does not*
06	Evaluation variables	*how the model grades itself*
07	Worked examples	*the same numbers worked through by hand*
08	Reference data & code map	*egg sizes, and where each rule lives*
09	Caveats & inconsistencies	*three things worth knowing before you trust a number*

Use the PDF bookmark panel to jump between sections.

01 How the whole thing works

One idea powers the whole system: **find past days that looked like today, and average what you sold on them.** Everything else — the confidence badge, the restock warning, the accuracy gauge — is built on top of that single act of comparison.

1. **Describe today**

   The system builds a fingerprint of today: which weekday it is, whether it is a weekend or a Philippine holiday, and how much demand the last 7, 14 and 30 days averaged, plus whether demand is rising or falling. That fingerprint is **7 numbers** — the independent variables.

2. **Find the look-alikes**

   Every past sales day for that egg size — up to **365 days** of history — gets the same fingerprint. Each one is compared with today using a distance formula, and the closest few are kept. Nothing is scored on days after the day being predicted.

3. **Average them, weighted**

   The nearest **k** days are averaged. Closer days count more (`1/√distance`), and older days fade further (`e^(-daysAgo/τ)`). The model picks *k* itself by trying 3, 5, 7 and 9 and keeping whichever scores best in a walk-forward test.

4. **Convert to eggs, money, and a confidence band**

   The tray average is rounded once, then multiplied by that size's tray capacity to get eggs. Money is forecast by a **second, independent pass** over the same days. A 90% bootstrap band is drawn around the tray number, and a confidence level is assigned from how many past days were available.

5 **Compare against the shelf and give a verdict**

Expected eggs are subtracted from eggs actually in stock. The result decides one of three verdicts — **Restock now**, **Order more**, or **Enough in stock** — and separately decides whether a low-stock alert is raised. Separately again, each tray gets an **Urgency** tag from its expiry date.

> **THE SINGLE MOST IMPORTANT IDEA**
>
> The forecast and the stock check are two different questions. The forecast asks *"how many eggs will I sell today?"* The verdict asks *"do I have that many?"* A high forecast with low stock produces **Restock now**; a low forecast with plenty of stock produces **Enough in stock**. Neither number alone means anything.

Every figure quoted in this document is read directly from the source files listed in Section 08. Where the code and its own comments disagree, the disagreement is recorded in Section 09.

02 The five classifications & their thresholds

Five separate labels are applied to stock and forecasts. They are computed in different files and do not talk to each other, which is why their cut-off values differ. All values below are hard-coded unless marked configurable.

2.1 Low Stock – `classes/AlertManager.php:84`

Raised as an alert on the dashboard. The idea is simple: *if what I have is less than a fixed fraction of what I am likely to need, warn me.* The default fraction is one half.

SETTING / CONDITION	VALUE	MEANING
`low_stock_threshold`	**0.5** configurable	The ratio of supply to demand below which you are warned. Stored in `system_settings`.
– clamped range	**0.1 – 1.0**	Whatever is entered, the value is forced inside this range. 1.0 means "warn only when demand exceeds *all* your stock".
Reference demand	**last 7 days, else forecast**	Prefer real sales from the previous 7 days. If a size had no sales, fall back to its average forecast for the last 7 days.
Fires when – condition A	**stock = 0**	Nothing left at all. Always fires, regardless of demand.
Fires when – condition B	**stock < demand × 0.5**	You hold less than half of what is expected to move.
Fires when – condition C	**stock < recentSales × 0.5**	Same idea, but benchmarked against what actually sold in the last 7 days.
Severity – out of stock	HIGH	Stock is zero. Message: "Out of stock — nothing left to sell".
Severity – low stock	MEDIUM	Stock is positive but under the ratio. Message states how many eggs are left and what they were compared against.

> **WHY ALERTS DO NOT PILE UP**
>
> Low-stock alerts are **reconciled**, not appended. Every run compares the currently-true alerts against the alert table: anything still true is kept, anything no longer true is marked resolved, and anything new is raised. One row per egg size, never one per tray.

2.2 Sufficient Stock – `pages/predictions.php:286`

The green banner at the top of the Sales Forecast page. It is the *absence* of a shortage, so its rule is a pair of conditions rather than a single number.

CONDITION	VALUE	MEANING
`No shortages`	**list is empty**	For every egg size with a forecast, `predicted_eggs − stock_eggs ≤ 0`.
`AND forecast is real`	**totalTrays > 0**	Stops the banner claiming success when there is nothing to sell at all.
`Shortage definition`	**shortage = max(0, predicted_eggs − stock_eggs)**	Eggs, not trays. Negative results are floored at zero, so surplus never counts as a shortage.
`trays_to_buy`	**ceil(shortage ÷ capacity)**	Converted to whole trays to buy. Capacity is floored at 1 to avoid divide-by-zero.
`Ordering of the list`	**largest shortage first**	Sizes are sorted so the biggest gap appears at the top.

2.3 Verdict — "Enough in stock" – `includes/functions.php:294`

One verdict per egg size, shown in the last column of the forecast table. It is computed from the same shortage figure but reported as a single tone, then mapped to a phrase.

VERDICT SHOWN	TONE	EXACT RULE	PLAIN MEANING
Restock now	danger	`stockEggs ≤ 0`	You have none of this size. Nothing can be sold until you restock.
Order more	warning	`stockEggs > 0 AND predicted_eggs − stockEggs > 0`	You have some, but not enough to cover what is expected. The sentence states the shortfall.
Enough in stock	success	`stockEggs − predicted_eggs ≥ 0`	Stock covers the forecast. Any remainder is reported as spare eggs.
—	muted	`is_no_data = true`	Too little sales history to forecast at all. Prompted to record a few days of sales.

> **THE TWO GREEN LABELS ARE NOT THE SAME RULE**
>
> **Sufficient Stock** (the banner) is all-or-nothing across every size: if even one size is short, the banner disappears. **Enough in stock** (the table column) is judged per size. It is perfectly normal to see the banner absent while several rows still read "Enough in stock".

2.4 Urgency – `pages/picklist.php:40`

A tag on every tray in the FIFO pick list. It answers *"how soon must this specific tray move?"* and is based purely on dates, never on demand.

TAG	CSS CLASS	RULE	WHY IT MATTERS
EXPIRED	`urgency-expired`	`days_until_expiry < 0`	Past its expiry date. Greyed out — do not sell.
SELL NOW	`urgency-high`	`days_until_expiry ≤ 3`	Highest risk of waste. Pick first.
USE SOON	`urgency-medium`	`days_until_expiry ≤ 7`	Still sellable, but plan it into the week.
AGING	`urgency-low`	`days_in_storage ≥ 14`	Plenty of time left, but it has been sitting two weeks or more. Rotate it.
no tag	`(empty)`	`otherwise`	Fresh and comfortably within shelf life.

Rules are applied in order and the first match wins, so **EXPIRED** *always beats the others, and* **AGING** *can only appear once expiry is more than 7 days away.*

A second, separate set of expiry rules drives the dashboard *expiring* alerts. These use a configurable look-ahead window rather than the fixed 3 / 7 day cut-offs above:

SETTING / CONDITION	VALUE	SEVERITY
`alert_expiry_days`	**3** configurable**, minimum 1**	Look-ahead window for scanning trays
`soonest < 0`	**already expired**	HIGH
`soonest = 0`	**expires today**	HIGH
`soonest = 1`	**1 day left**	HIGH
`soonest = 2 or 3`	**inside the window**	MEDIUM
`Grouping`	**one row per egg size**	Not one row per tray — 270 trays expiring together is one decision, not 270 pieces of news.

2.5 Forecast Reliability

There is no single "reliability" number. Reliability is expressed twice: once as a **confidence level** on each individual forecast, and once as an **accuracy gauge** for the model as a whole. They answer different questions — "can I trust *this* number?" versus "can I trust *the model* in general?".

2.5A CONFIDENCE LEVEL — THE "SURE" COLUMN – KNNPREDICTOR.PHP:353

Driven purely by *how many past days the model had to learn from*. More history, more trust. A stale size is automatically downgraded one step.

LEVEL	EFFECTIVE SAMPLE THRESHOLD	MEANING
High	`≥ 30 days`	A solid month-plus of history. Reasonable to plan stock around it.

LEVEL	EFFECTIVE SAMPLE THRESHOLD	MEANING
Medium	10 – 29 days	Enough to be directionally useful; expect misses of a tray or two.
Low	< 10 days	Thin history. Treat the number as a rough guide.
None	< 7 rows of history	No forecast is produced at all. The page shows a prompt to record sales.

THE DOWNGRADE RULE

SETTING / CONDITION	VALUE	EFFECT
days_since_last_sale > 30	**30 days**	The size's own sales are too old to be representative.
→ effectiveSamples	**min(sampleCount, 8)**	Capping at 8 forces the level to Low no matter how much history exists.
UI signal	**"cut" + gap in days**	The Sure column marks the badge as cut and shows how many days since that size last sold.

Why this exists: after a 30-day gap the 7-, 14- and 30-day demand windows all fall to zero, so the model can only match on weekday. Sample count alone would still say "High", which would overstate how much the number is worth.

THE CONFIDENCE INTERVAL AROUND THE NUMBER

PARAMETER	VALUE	MEANING
Resamples	**100**	Bootstrap rounds used to build the band.
Percentiles used	**5th & 95th**	The band spans 90% of resampled outcomes.
Minimum half-width	**0.5 trays**	A band narrower than half a tray is widened to it — the model never claims false precision.
n < 5 fallback	**prediction × 0.5**	Too few samples to bootstrap, so a crude 50% band is used instead.

2.5B ACCURACY GAUGE — MODEL HEALTH – MODELEVALUATOR.PHP:253,310

BAND	OVERALL F1 SCORE	COLOUR & VERDICT
Excellent	≥ 90	GREEN Safe to plan stock around the forecast
Good	80 – 89	GREEN
Fair	70 – 79	YELLOW Watch it

BAND	OVERALL F1 SCORE	COLOUR & VERDICT
Weak	`60 – 69`	YELLOW
Poor	`< 60`	RED Retrain or reconsider

Individual metric rows use a coarser three-band rule against a **90%** target:

VERDICT	METRIC BAND	RULE
Good	`≥ 90`	Meets the target outright.
Watch	`76.5 – 89.9`	85% of the target. Close enough to monitor, not to trust.
Weak	`< 76.5`	Materially below target.

76.5 is not a typed-in number — it is `threshold × 0.85` = `90 × 0.85`. *If the target is ever changed, both boundaries move with it.*

AUTOMATIC MODEL-HEALTH ALERTS

METRIC	ALERT BELOW	SEVERITY
Accuracy	`< 80%`	HIGH
Recall	`< 80%`	HIGH
Precision	`< 75%`	MEDIUM
F1 score	`< 75%`	MEDIUM
Minimum samples	`≥ 5 required`	Sizes with fewer scored days are skipped entirely.

Source: `AlertManager.php:151`

DATA FRESHNESS BANNER

A separate global check runs before the per-size confidence, so the page can warn when *no* size has recent sales. Because the predictor already downgrades per size, this banner does not reduce confidence a second time — it only explains why.

LABEL	DAYS SINCE LAST SALE	WORDING
Up to date	`≤ 2`	Nothing to flag.
A few days behind	`3 – 7`	Normal.
Behind by N days	`8 – 30`	Recent demand windows still meaningful.
Behind by N days	`> 30`	Forecast can only match on weekday. Banner shown; per-size confidence already cut.

Source: `pages/predictions.php:31–48`

03 Independent variables — the inputs

These are the things the model is allowed to *look at*. They are the independent variables: if you changed one of them, the forecast would change. There are exactly **seven**, declared in one place, and no others are used.

THE COMPLETE LIST

`day_of_week` · `is_weekend` · `is_holiday` · `demand_7d` · `demand_14d` · `demand_30d` · `demand_trend`

Critically: *no inventory variable is an input. Trays in stock, expiry dates and quantity levels are deliberately excluded from the forecast so that the model predicts demand, never supply. Stock only enters afterwards, when forming the verdict.*

3.1 Calendar variables

`day_of_week`

Which day of the week is it? For an egg shop, Saturday is not Tuesday.

PHP's `date('w')`, giving **0 = Sunday through 6 = Saturday**. This is usually the single strongest signal: weekends carry heavier retail traffic than midweek.

It is stored in the database as 1–7 using `DAYOFWEEK()`, which subtracts 1 to realign it with the 0–6 scale the model uses.

Range 0–6 · Type: integer · Source: target date

`is_weekend`

Is today Saturday or Sunday?

Simply `1` when `day_of_week` is 0 or 6, otherwise `0`. This is deliberately redundant with `day_of_week` — it is a derived convenience flag that makes the weekend effect explicit in the distance calculation rather than something the model has to infer from seven separate numeric distances.

Two features describing the same fact is a mild form of double-counting. It is kept because the normalised distance for `day_of_week` treats "Sunday to Tuesday" as merely two steps away, which understates how different they really are.

Range 0–1 · Type: integer · Derived from day_of_week

`is_holiday`

Is today a Philippine public holiday?

Flag `1` if the date appears in the generated holiday list, otherwise `0`. The list has two parts:

- **9 fixed dates** — 1 Jan, 9 Apr, 1 May, 12 Jun, 21 Aug, 1 Nov, 30 Nov, 25 Dec, 30 Dec.
- **3 movable dates** — Maundy Thursday, Good Friday and Easter Sunday, all derived from a computable Easter algorithm rather than a hard-coded calendar, so the dates are always right.

Holiday egg sales behave differently enough to be worth flagging separately from "weekend".

Range 0–1 · Type: integer · Source: `functions.php:170`

3.2 Demand-window variables

*All three use the same query: the average trays sold per recorded day over a window that ends **strictly before** the target date. Two details matter. First, the window is `date < target`, so today's partial sales never leak into today's forecast. Second, it averages over recorded days, not calendar days — so one quiet Tuesday does not drag the average down as hard as it would under a calendar-day denominator. With `COALESCE(..., 0)`, a size with no rows in the window returns `0` rather than `NULL`.*

`demand_7d`

On a typical recent day, how much did I sell?

Mean `trays_sold` across the previous 7 days. This is the model's main sense of *current* demand level and typically carries the most weight in the distance calculation, because it is the hardest of the seven features to accidentally match.

Unit: trays/day · Type: float · Window: 7 days

`demand_14d`

What about the last two weeks?

The same average over 14 days. Where `demand_7d` is a fast read, this one is steadier and partly smooths out a single unusual day. Where the two disagree, demand is either climbing or falling — which is what the trend feature formalises.

Unit: trays/day · Type: float · Window: 14 days

`demand_30d`

And over a full month?

The 30-day average. This is the slowest and most stable of the three. It is what keeps a fortnight of unusually good trading from being read as a permanent new normal — a past day whose 30-day demand looks nothing like the current month will correctly rank as a poor match.

Unit: trays/day · Type: float · Window: 30 days

`demand_trend`

Is demand rising or falling? This week versus the week before.

Not a raw figure but a **difference**:

```
demand_trend = demand_7d  −  average(trays sold over days 8–14 ago)
```

Positive means this recent week beat the one before; negative means it fell short. Including the slope as well as the level lets the model prefer a rising Tuesday over a flat one even when both average the same number of trays.

Unit: trays/day difference · Type: float · Can be negative

3.3 How the seven become a prediction

The seven features are only comparable because they are put on a common scale first. For every training set, each feature's own minimum and maximum are recorded and every value is rescaled to fall between 0 and 1. Without this, `day_of_week` (0–6) would dominate the arithmetic simply by being numerically larger than `is_holiday` (0–1).

DISTANCE & NEIGHBOUR SELECTION

Each training day's normalised fingerprint is compared with today's by straight-line (Euclidean) distance across all seven features — the square root of the summed squared gaps. Days are then sorted by distance and the nearest ***k*** are kept.

WEIGHTING & AVERAGING

Each kept day gets a weight of `1/√distance`, multiplied by a recency factor of `e^(-daysAgo / τ)`. The prediction is the weighted mean of those days' sales.

> **WHY 1/√DISTANCE INSTEAD OF THE TEXTBOOK 1/DISTANCE**
>
> Plain `1/d` collapses almost all the weight onto the single nearest day, so *k* quietly behaves like "nearest 1" with extra steps and the vote of the other neighbours is lost. `1/√d` still ranks nearer days first but lets the whole *k* window contribute. It is a deliberate departure from the textbook formula.

SUPPORTING VARIABLES THAT ARE NOT FEATURES

These shape the model but are not part of the distance calculation.

VARIABLE	VALUE / RANGE	ROLE
`egg_size`	`SS, SM, MD, LG, XL, JB, CR, SA`	Forecasts are entirely independent per size. Medium's history never informs Jumbo's.
`prediction_date`	`YYYY-MM-DD`	The day being predicted. The anchor for every window and for the strict "before" cut-off.
`k`	`3, 5, 7 or 9`	How many past days to average. Chosen per size by walk-forward test; forced to 3 below 14 history rows; default 5.
`tray_capacity`	`24 or 30 eggs`	Converts trays to eggs. Read per size from the database, falling back to 30.
`recency_tau (τ)`	`45 days`	Decay constant for the recency weight — the half-life of a day's influence. Overridable at runtime.
`training window`	`365 days`	Maximum history loaded per size, newest first.
`minimum history`	`7 rows`	Below this, no forecast is produced at all.

Note the two separate decay constants in the codebase: the live path uses `e^(-daysAgo/30)` and the backtest path uses `e^(-daysAgo/τ)` with τ = 45. See Section 09.

04 Dependent variables — the outputs

These are what the model produces. They depend on the seven inputs plus *k*. Unlike the inputs, these are the numbers you actually see and act on.

4.1 The two primary targets

The model is run **twice** over the same days and the same neighbours, once for each target. The two runs are independent — revenue is never calculated by multiplying a tray count by today's price.

TARGET	FITTED FROM	OUTPUT
actual_demand	`sales_history.trays_sold`	How many trays of this size will sell today.
actual_sales	`sales_history.total_amount`	How much money this size will take today.

> **WHY FORECAST REVENUE SEPARATELY?**
>
> Price changes over time and revenue per tray is not constant. Running a second pass over the same historical days lets the model learn what revenue each size *actually* returned. The consequence is deliberate and documented on the page: predicted sales will **not** exactly equal eggs × today's price, and that is expected rather than a bug.

4.2 Primary and derived outputs

OUTPUT	HOW IT IS PRODUCED	NOTES
`predicted_trays`	Weighted mean of the neighbours' `actual_demand`, rounded to a whole number.	The headline quantity. Clamped at ≥ 0.
`predicted_eggs`	`round(predicted_trays × tray_capacity)`	Derived from the *already-rounded* tray figure, not the raw one.
`predicted_sales`	Second weighted-mean pass over `actual_sales`, to 2 dp.	Currency.
`confidence_lower`	`prediction − interval`	Floored at 0.
`confidence_upper`	`prediction + interval`	90% bootstrap band.
`confidence_level`	Sample-count bands, after any staleness downgrade.	High / Medium / Low / None.
`confidence_raw_level`	Same bands, before the downgrade.	Kept so the UI can show that a cut happened.
`confidence_downgraded`	`true if the two levels differ`	Drives the “cut” note in the Sure column.
`k_used`	Walk-forward winner.	Reported so the choice is auditable.

OUTPUT	HOW IT IS PRODUCED	NOTES
`samples_used`	Count of training rows found.	Shown under the badge as “N days”.
`effective_neighbors`	Kish's effective sample size from the weight shares.	Reveals when the weight has collapsed onto effectively one day. Values near 1.00 mean the *k* window is doing less work than it appears.
`days_since_last_sale`	Days between the target and that size's most recent prior sale.	Drives the downgrade rule.

4.3 Derived comparison variables

These exist only to feed the verdict and the restock banner.

VARIABLE	FORMULA	PURPOSE
`stock_eggs`	`SUM(quantity) WHERE status = 'in_stock'`	What is on the shelf now, per size.
`stock_trays`	`COUNT(*) of in-stock trays`	Shown as a secondary line under the egg count.
`shortage`	`max(0, predicted_eggs − stock_eggs)`	Gap in **eggs**. Non-positive means covered.
`surplus`	`stock_eggs − predicted_eggs`	Spare eggs, reported when positive.
`trays_to_buy`	`ceil(shortage ÷ max(1, capacity))`	Whole trays to order for that size.
`tone`	Derived from stock_eggs and shortage.	`danger` / `warning` / `success` / `muted`
`total_trays / total_eggs / total_sales`	Summed across sizes with forecast data.	The hero figures. Floats are summed first, then rounded once at the end.

> **THE ROUNDING RULE THAT KEEPS TOTALS HONEST**
>
> Each size's tray figure is rounded **once**, then summed into its category total, then summed into the day total. Previously each size was truncated to an integer as it was added up, which lost up to a tray per size and made the visible totals disagree with each other. Eggs are always trays × capacity, so the two figures can never contradict each other.

4.4 No-data output

When fewer than 7 rows of history exist, the model refuses to guess. Rather than returning a misleading zero, it returns a flagged empty result with the counts needed to fix the problem.

FIELD	VALUE	MEANING
`is_no_data`	`true`	Suppresses every numeric output.
`predicted_trays / _eggs / _sales`	`null`	Rendered as an italic note, not as a 0.
`confidence_level`	`none`	Badge replaced with “Record sales”.
`message`	`text`	“No sales data available…” at 0 rows, or “Not enough data (N/7 samples)…” otherwise.
`samples_used`	`count`	How close the size is to the minimum.

05 Stock & lifecycle variables

These variables are **not** model inputs — the forecast never sees them. They are read straight from the inventory tables and used only by the threshold rules in Section 02. Listing them separately prevents the common mistake of treating them as predictive features.

VARIABLE	SOURCE	USED BY
`current_stock`	`SUM(egg_trays.quantity) WHERE status='in_stock'`	Low Stock, Verdict, Sufficient Stock
`tray_count`	`COUNT of in-stock trays`	Display under the egg figure
`tray_capacity`	`egg_sizes.tray_capacity`	Trays ↔ eggs conversion; shortage → trays to buy
`days_until_expiry`	`DATEDIFF(expiry_date, today)`	Urgency tag; expiring alerts
`days_in_storage`	`DATEDIFF(today, production_date)`	AGING urgency tag
`soonest`	`MIN(DATEDIFF(expiry_date, NOW())) per size`	Severity of the expiring alert
`recent_sold`	`SUM(receipt_items.quantity), 7-day, non-voided`	Low Stock reference demand
`avg_predicted_eggs`	`AVG(predicted_trays) × capacity, 7-day`	Low Stock fallback reference demand
`status`	`in_stock / sold / damaged / discarded / deleted`	Every stock figure filters on this
`fifo_priority_score`	`egg_trays column`	Pick-list ordering
`shelf_life`	`egg_sizes.shelf_life (3–45 days)`	Source of expiry dates

THE MOST IMPORTANT STRUCTURAL DECISION

Forecast and stock are kept strictly apart. A bug in stock tracking can therefore never silently corrupt the model, and the backtest scores can never be inflated by the model's own output being written back as "actual". This separation is what makes the accuracy figures trustworthy.

06 Evaluation variables

These are also dependent variables — they depend on the model's predictions — but they are computed afterwards, by replaying the forecast over real past days. They tell you how much the whole system is worth trusting.

> **SCORED BY BACKTEST, NOT BY READING STORED ROWS**
>
> Metrics come from replaying the forecast across past days, where each day is predicted using **only days before it**. The stored `knn_predictions` table is never used as the scoring source, because it can be seeded, stale, or partly written by a cron job — and because the model itself writes to it. Anything less than 5 scored days returns an empty result set.

6.1 Tolerance metrics closeness

METRIC	FORMULA	WHAT IT MEANS
`accuracy`	`(days within ±1 tray ÷ total) × 100`	Hit rate. One tray is accepted as a miss-free day because a hand-counted tray is rarely exact.
`exact_accuracy`	`(days within ±0.5 tray ÷ total) × 100`	Stricter version — effectively an exact match on the rounded tray count.
`over_predicted`	`count where (predicted − actual) > 0.5`	Days the model guessed too high.
`under_predicted`	`count where (predicted − actual) < −0.5`	Days the model guessed too low.

6.2 Directional metrics bias, not closeness

METRIC	FORMULA	WHAT IT MEANS
`precision`	`((total − over_predicted) ÷ total) × 100`	**Guards your cash.** Share of days the model did not over-predict. Over-forecasting ties up money in stock that then ages.
`recall`	`((total − under_predicted) ÷ total) × 100`	**Guards your sales.** Share of days the model reached at least the actual figure. Under-forecasting turns customers away.
`f1_score`	`2 × (P × R) ÷ (P + R)`	Harmonic mean. Falls sharply when *either* direction is unhealthy, which is why it is used for the headline gauge.

> **THESE ARE NOT CLASSICAL PRECISION AND RECALL**
>
> There is no true/false-positive matrix here. Both figures reduce to the **sign** of `(predicted − actual)` against a ±0.5-tray cut, so they measure *directional bias*, not classification quality. They are still useful — a model that

is consistently 2 trays high will score badly on precision even if its average error is small — but the conventional textbook definitions do not apply. The page states this explicitly in its own footnote.

6.3 Error metrics magnitude

METRIC	FORMULA	WHAT IT MEANS
`mae`	`mean(|actual − predicted|)`	Average miss in trays. The most readable of the error figures.
`mse`	`mean((actual − predicted)²)`	Squares the errors, so outliers dominate.
`rmse`	`√mse`	Same units as MAE, outlier-sensitive. Drives the "Worst-Day Miss" figure.
`mape`	`mean(|error| ÷ actual) × 100`	Percentage error. Skipped whenever actual is 0.
`r2`	`1 − (Σresidual² ÷ Σtotal²)`	Share of day-to-day variation explained. Bands: ≥0.75 green, ≥0.50 yellow, else red.
`adj_r2`	`r2 penalised for 7 features`	Adjusted R². Falls back to plain r2 when there are too few samples (< 8).

6.4 Bookkeeping variables

VARIABLE	MEANING
`sample_count`	Number of scored size-by-day checks. Runs several times higher than the number of calendar days, since every size is scored every day.
`confusion_matrix`	Bundle of within_tolerance, exact_match, over_predicted, under_predicted, total.
`evaluation_window`	First and last scored date. Feeds the "between X and Y" wording.
`evaluation_source`	Always `walk_forward_backtest`.
`calculation_formula`	The arithmetic written out in full, shown per size on request.

WHY THE HEADLINE GAUGE USES F1, NOT ACCURACY

Accuracy on its own is misleading here, because the ±1-tray tolerance is generous. A model that is systematically short can post a very high accuracy while its recall sits near 50%. F1 falls when either direction is unhealthy, so it is the safer single number to headline. Accuracy is still reported alongside it.

07 Worked examples

The same numbers pushed through each rule by hand, to check the thresholds behave as documented.

7.1 A forecast, from raw neighbours to eggs

Medium (MD) today. Tray capacity 30. The three nearest past days were:

PAST DAY	DISTANCE	1/√D	RECENCY E^(−D/T)	WEIGHT	TRAYS SOLD
3 days ago	0.05	4.47	0.936	4.18	7
10 days ago	0.40	1.58	0.801	1.27	5
24 days ago	0.90	1.05	0.587	0.62	4

```
weightedSum   = 4.18×7 + 1.27×5 + 0.62×4 = 29.26 + 6.35 + 2.48 = 38.09
weightTotal   = 4.18 + 1.27 + 0.62                              = 6.07

predicted_trays = round(38.09 ÷ 6.07) = round(6.28)      = 6 trays
predicted_eggs  = round(6 × 30)                          = 180 eggs
```

Note the rounding order: eggs come from the rounded tray count. Deriving eggs from the raw 6.28 would give 188 eggs, producing the impossible pairing of "6 trays / 188 eggs" for a 30-egg tray.

7.2 Confidence level and the staleness cut

```
Scenario A — Medium has 42 days of history, last sale 5 days ago
  effectiveSamples = 42  →  ≥ 30  →  HIGH          (no cut)

Scenario B — Medium has 42 days of history, last sale 44 days ago
  gapDays = 44 > 30  →  effectiveSamples = min(42, 8) = 8
  8 < 10  →  LOW                              (cut from High)
  confidence_raw_level = HIGH  →  the "cut" note appears, showing "44d since last sale"

Scenario C — Medium has 5 days of history
  5 < 7  →  no forecast at all  →  confidence_level = NONE, is_no_data = true
```

Scenario B is the important one: the size has plenty of history, but all of it is too old to be representative, so the badge is cut anyway.

7.3 Verdict and the banner

SIZE	PREDICTED EGGS	IN STOCK	SHORTAGE	VERDICT
Medium	180	240	0	Enough in stock *60 spare*
Large	300	210	90	Order more *3 trays to buy*
Jumbo	48	0	48	Restock now

*Banner outcome: **"Restock before you sell"** — not "Stock looks sufficient", because the banner requires zero shortages across every size, and two sizes are short. This is the difference between the banner and the column.*

7.4 Low Stock with a custom threshold

```
Salted (SA): in stock 150 eggs · sold 400 eggs in the last 7 days

default 0.5 →  400 × 0.5 = 200  →  150 < 200  →  LOW STOCK, medium
threshold 0.7 →  400 × 0.7 = 280  →  150 < 280  →  LOW STOCK, medium
threshold 0.3 →  400 × 0.3 = 120  →  150 > 120  →  no alert

No sales in 7 days → falls back to forecast: 180 eggs × 0.5 = 90 → 150 > 90 → no alert
```

7.5 Urgency from two dates

TRAY	PRODUCED	EXPIRES	DAYS TO EXPIRY	DAYS IN STORAGE	TAG
A	1 Jun	10 Jun	−2	4	EXPIRED
B	20 May	8 Jun	3	14	SELL NOW
C	20 May	13 Jun	8	14	AGING
D	5 Jun	26 Jun	21	8	no tag

Trays B and C are the instructive pair: identical age in storage, but B expires in 3 days so the expiry rule fires first. Age only decides the tag once expiry is more than 7 days away.

7.6 Metric status boundaries

```
Measured F1 = 91.0%   →  ≥ 90.0  →  GREEN — Good
Measured F1 = 89.9%   →  < 90.0 but ≥ 76.5  →  YELLOW — Watch
Measured F1 = 76.5%   →  exactly at the boundary  →  YELLOW — Watch
Measured F1 = 76.4%   →  below 76.5  →  RED — Weak
Measured F1 = 45.0%   →  gauge band "Poor", colour RED
Measured F1 = 0 samples →  grey — insufficient data
```

7.7 Accuracy on a single backtested day

```
Actual sold 6 trays · predicted 5 trays
  error = 1  →  within ±1 tray  →  accuracy counts this day as a hit
  within ±0.5? No → counts as an over/under day (under, by 1 tray)

Actual sold 6 trays · predicted 7 trays
  error = 1  →  hit on accuracy, but sign is positive → over-predicted → costs precision

Actual sold 0 trays · predicted 2 trays
```

```
error = 2  →  miss on accuracy, over-predicted → costs precision
mape skips this day (division by zero protected)
```

08 Reference data & code map

8.1 Egg sizes – `egg_sizes`

CODE	LABEL	CAPACITY	PRICE / TRAY	PRICE / EGG	SHELF LIFE	CATEGORY
`SS`	Super Small	30	₱180.00	₱6.00	21 days	fresh
`SM`	Small	30	₱210.00	₱7.00	21 days	fresh
`MD`	Medium	30	₱240.00	₱8.00	21 days	fresh
`LG`	Large	30	₱270.00	₱9.00	21 days	fresh
`XL`	Extra Large	24	₱240.00	₱10.00	21 days	fresh
`JB`	Jumbo	24	₱288.00	₱12.00	21 days	fresh
`CR`	Cracked	30	₱120.00	₱4.00	3 days	cracked
`SA`	Salted	30	₱360.00	₱12.00	45 days	salted

*Tray capacity is **not uniform** — XL and JB hold 24 eggs while the rest hold 30. Any code that assumes 30 eggs per tray is wrong for those two sizes. The application falls back to 30 when a size has no capacity recorded.*

8.2 Configurable settings – `system_settings`

KEY	DEFAULT	EFFECT
`low_stock_threshold`	`0.5`	Supply-to-demand ratio below which Low Stock fires. Clamped 0.1–1.0.
`alert_expiry_days`	`3`	Look-ahead window for expiry scanning. Minimum 1.
`auto_retrain`	`1`	Whether the weekly retrain runs automatically. Checked on every forecast read.
`timezone`	`Asia/Manila`	All date arithmetic.

8.3 Where each rule lives

RULE	LOCATION
Feature list (the 7 inputs)	`KNNPredictor.php:25`
Feature extraction	`KNNPredictor.php:103`
Demand window queries	`KNNPredictor.php:125, 134`
Min-max normalisation	`KNNPredictor.php:172`

RULE	LOCATION
Weighted KNN (live)	`KNNPredictor.php:184`
Weighted KNN (backtest)	`KNNPredictor.php:817`
Euclidean distance	`KNNPredictor.php:235`
k selection (3/5/7/9)	`KNNPredictor.php:262`
Walk-forward validation	`KNNPredictor.php:298`
Bootstrap confidence band	`KNNPredictor.php:327`
Confidence level bands	`KNNPredictor.php:353`
Staleness downgrade	`KNNPredictor.php:56`
No-data result	`KNNPredictor.php:359`
Backtest series	`KNNPredictor.php:624`
Egg → tray conversion	`KNNPredictor.php:70, 1018`
All backtest metrics	`ModelEvaluator.php:61`
Accuracy / precision / recall / F1	`ModelEvaluator.php:143–151`
R^2 and adjusted R^2	`ModelEvaluator.php:99–120`
Overall gauge bands	`ModelEvaluator.php:310`
Metric status bands	`ModelEvaluator.php:253 / functions.php:278`
Low Stock rule	`AlertManager.php:84`
Expiring alert severity	`AlertManager.php:30`
Model-health thresholds	`AlertManager.php:151`
Verdict text	`functions.php:294`
PH holiday calendar	`functions.php:170`
Freshness banner	`pages/predictions.php:31`
Sufficient Stock banner	`pages/predictions.php:286`
Urgency tags	`pages/picklist.php:40`

09 Caveats & inconsistencies

Three places where the code does not fully agree with itself. None breaks the system today, but each one is worth knowing before a number is treated as authoritative.

1 TWO DIFFERENT CONFIDENCE-BAND RULES EXIST

The canonical `getConfidenceLevel()` uses **≥30 → High, ≥10 → Medium, else Low**, and this is what the live forecast and the Sure column use.

A separate inline rule elsewhere in the same file uses **≥30 → High, ≥14 → Medium, else Low** when writing confidence during retraining.

Effect: a size with 10–13 history days can display *Medium* on the forecast page while having been persisted as *Low*. The displayed value is the more generous of the two. The two should be reconciled to one shared function.

2 TWO DIFFERENT RECENCY DECAY CONSTANTS EXIST

The live prediction path decays by `e^(-daysAgo/30)`, giving a 30-day time constant. The backtest path uses `e^(-daysAgo/τ)` with **τ = 45**, overridable via `setRecencyTau()`.

Effect: the live forecast and the backtest that grades it weight recency differently. The reported accuracy therefore measures a slightly different model from the one that produces the daily number. Using one constant in both places would make the score strictly honest.

3 PRECISION AND RECALL ARE SIGN TESTS, NOT CLASSIFICATION SCORES

As covered in Section 06.2, both reduce to the sign of the error against a ±0.5-tray cut. A model that over-predicts every single day by one tray would score **0% precision** while having a perfectly respectable hit rate — which is informative, but is not what the textbook terms imply to anyone reading the table without the footnote.

Recommendation: keep the metrics, but consider renaming them to "Did not over-predict" / "Did not under-predict" everywhere, so the label and the maths agree.

Lower-risk observations

- `is_weekend` **duplicates** `day_of_week`**.** Both contribute to the distance, so weekends are effectively weighted double. Defensible as an explicit interaction term, but it is undocumented as one.
- **No minimum sample count for** `metricStatus()` **or the gauge bands.** A size with 5 scored days at 100% shows the same green as one with 200 scored days at 90%. The `sample_count ≥ 5` filter exists for model-health alerts but not for the on-page metric rows.
- **Forecast page shows one global staleness banner but per-size confidence.** These are two separate mechanisms. The banner is global (any sales activity at all), the downgrade is per size. The page correctly avoids double-counting, but the two can disagree on screen.
- **Low Stock compares against a 7-day average forecast** when a size had no sales, which can alert on a size the model itself rates Low confidence. The message does not surface the confidence level.

WHAT THE DESIGN GETS RIGHT

Three decisions are worth protecting in any future change, because each one fixes a real failure mode:

1. **Forecast and stock are fully separated.** Stock levels never enter the model, so inventory bugs cannot silently corrupt demand predictions.
2. **Metrics are scored by walk-forward backtest, not from stored rows.** The model can never be graded against a figure it helped produce.
3. **Confidence is downgraded on staleness, not just sample count.** Plenty of old data is treated as worse than a modest amount of fresh data, which is the honest relationship.

Thresholds transcribed from source on the date shown on the cover. Behaviour at exact boundary values is noted where the comparison operator is inclusive.